

1 Navigation2 概述

什么是Navigation2?

Navigation2 (Nav2) 是ROS2的官方导航框架，是ROS1中move_base的继承者和大幅改进版本。它为移动机器人提供完整的自主导航解决方案。

主要特点

- ✓ **模块化设计** - 各组件独立，可自由组合
- ✓ **行为树架构** - 复杂任务的状态管理
- ✓ **多种算法支持** - A、*Theta*、RRT*等路径规划算法
- ✓ **动态重配置** - 运行时参数调整
- ✓ **强大的恢复机制** - 智能故障处理
- ✓ **完整的3D支持** - 支持3D环境导航

适用场景

- 🏠 **室内服务机器人** - 家庭、办公室导航
- 🏭 **工业AGV** - 仓储、工厂自动化
- 🚗 **自动驾驶** - 低速园区车辆
- 🤖 **研究平台** - 学术研究和原型开发

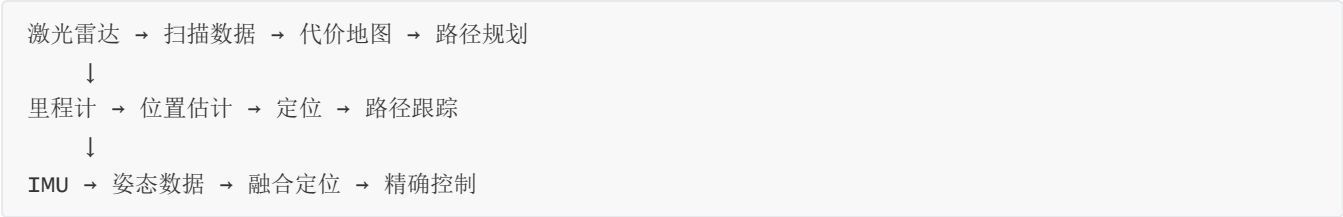
2 核心概念理解

2.1 坐标系统 (Coordinate Frames)

Navigation2使用多个坐标系来描述机器人的位置和环境：

坐标系	用途	特点
map	全局固定坐标系	地图的原点，不会移动
odom	里程计坐标系	基于轮式编码器，会累积误差
base_link	机器人本体坐标系	机器人的几何中心
base_footprint	机器人投影坐标系	base_link在地面的投影

2.2 传感器数据流



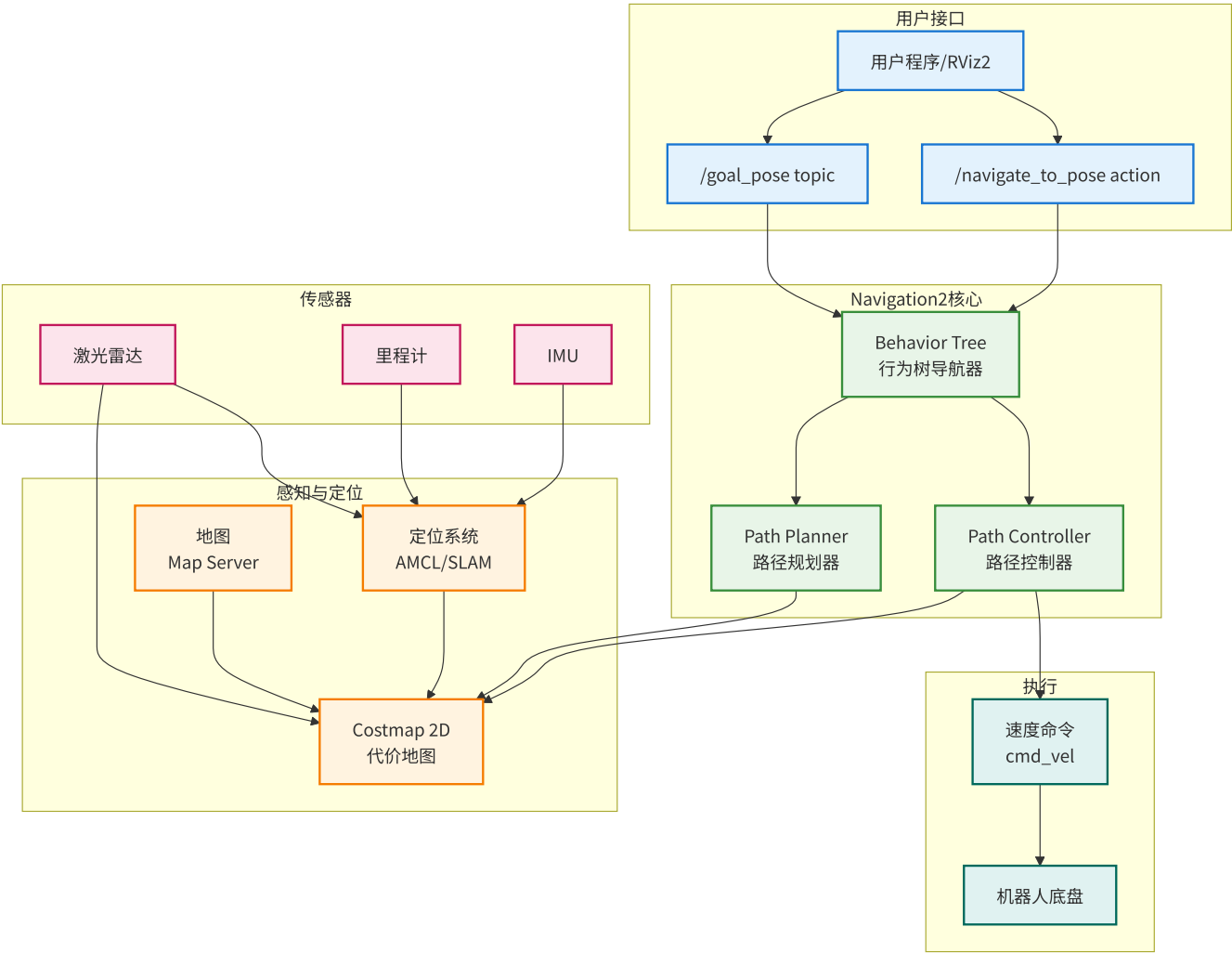
IMU 是[惯性测量单元](#)（Inertial Measurement Unit）的缩写。它是一种电子设备，用于测量和报告物体的三个基本线性运动（加速度）和三个基本角运动（角速度）。IMU无论大小从超小型的MEMS传感器，到高精度的激光陀螺，其原理都是采用惯性定律实现的，IMU 通常包括一组加速度计和[陀螺仪](#)，他们是惯性系统的核心部件，是影响惯性系统性能的主要因素。此外，有时还包括[磁力计](#)。

2.3 路径规划层次

- **全局规划器 (Global Planner):** 从起点到终点的长期路径
- **局部规划器 (Local Planner):** 实时避障和轨迹跟踪
- **恢复行为 (Recovery Behaviors):** 遇到困难时的恢复策略

3 系统架构详解

3.1 Navigation2完整架构图



3.2 核心服务器组件

上图展示了Navigation2的完整架构，包含以下核心组件：

组件	功能	作用
BT Navigator	行为树导航器	整个导航系统的大脑，协调各个组件
Planner Server	路径规划服务器	全局路径规划，从起点到终点
Controller Server	控制器服务器	局部路径跟踪和实时避障
Recovery Server	恢复行为服务器	处理导航失败的恢复策略

3.2.1 BT Navigator (行为树导航器)

主要作用：

- 作为导航系统的**主控制器**，协调所有其他导航组件
- 使用行为树（Behavior Tree）来管理复杂的导航逻辑
- 负责处理高级导航策略和异常情况
- 接收导航目标并编排整个导航流程

核心功能：

- 接收 `NavigateToPose` 动作请求
- 编排路径规划、路径跟踪、恢复行为的执行顺序
- 处理导航过程中的异常和失败情况
- 提供灵活的导航行为配置

3.2.2 Planner Server (路径规划服务器)

主要作用：

- 负责**全局路径规划**，计算从起点到终点的最优路径
- 基于全局代价地图进行路径搜索
- 提供不同的路径规划算法插件

核心功能：

- 接收 `ComputePathToPose` 服务请求
- 使用全局代价地图计算无碰撞路径
- 支持多种算法：A*、Dijkstra、NavFn等
- 考虑机器人几何形状和运动约束

3.2.3 Controller Server (控制器服务器)

主要作用：

- 负责**局部路径跟踪**和实时运动控制
- 将全局路径转换为具体的速度指令
- 处理动态障碍物避障

核心功能：

- 接收 `FollowPath` 动作请求
- 基于局部代价地图进行实时路径调整
- 生成安全的速度指令（`cmd_vel`）
- 支持多种控制算法：DWB、TEB、Pure Pursuit等

3.2.4 Recovery Server (恢复服务器)

主要作用：

- 当导航遇到困难时执行**恢复行为**
- 帮助机器人脱离困境，继续完成导航任务
- 提供多种恢复策略

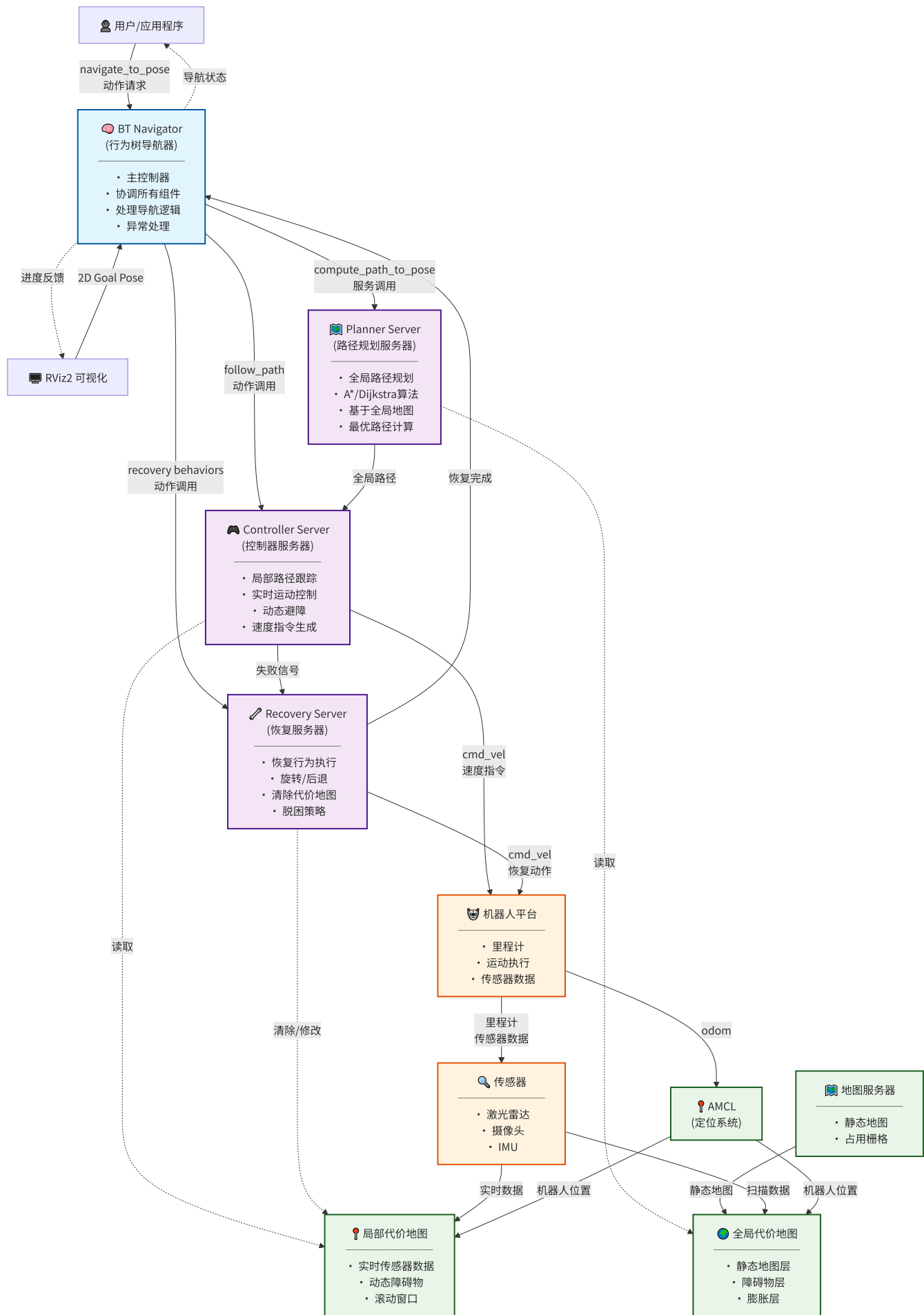
核心功能：

- 执行旋转、后退、清除代价地图等恢复动作
- 处理机器人被卡住或路径受阻的情况
- 支持自定义恢复行为插件
- 与BT Navigator协调执行恢复序列

3.3 详细框架和流程

3.3.1 组件之间的关系

详细的框架图来展示这些组件之间的关系：



3.3.2 详细工作流程

1. 导航流程 (典型情况)

用户设置目标 → BT Navigator → Planner Server → Controller Server → 到达目标

1. 用户发送导航目标 (navigate_to_pose)
2. BT Navigator 接收请求，开始执行行为树
3. 调用 Planner Server 计算全局路径
4. 调用 Controller Server 跟踪路径并生成速度指令
5. 持续监控 导航进度和异常情况

2. 异常处理流程

导航失败 → BT Navigator → Recovery Server → 重新尝试导航

当遇到以下情况时会触发恢复行为：

- 机器人被卡住
- 路径被阻挡
- 控制器失败
- 规划器找不到路径

3. 从配置文件看到的具体实现

从您的项目配置中可以看到：

BT Navigator 配置：

```
bt_navigator:
  plugin_lib_names:
    - nav2_compute_path_to_pose_action_bt_node # 调用路径规划
    - nav2_follow_path_action_bt_node         # 调用路径跟踪
    - nav2_back_up_action_bt_node             # 后退恢复
    - nav2_spin_action_bt_node                # 旋转恢复
```

Planner Server 配置：

```
planner_server:
  planner_plugins: ["GridBased"]
  GridBased:
    plugin: "nav2_navfn_planner/NavfnPlanner" # 使用NavFn算法
```

Controller Server 配置：

```
controller_server:
  controller_plugins: ["FollowPath"]
  FollowPath:
    plugin: "dwb_core::DWBLocalPlanner"    # 使用DWB控制器
```

Recovery Server 配置：

```
behavior_server:
  behavior_plugins: ["spin", "backup", "drive_on_heading", "wait"]
```

3.4 算法插件

全局规划器选项:

- **NavFn** - 经典的导航函数算法
- **A*** - A星算法，平衡效率和最优性
- **Theta*** - A*的变种，支持any-angle路径
- **RRT*** - 快速随机探索树算法

局部规划器选项:

- **DWB (Dynamic Window Branch)** - 动态窗口方法
- **TEB (Timed Elastic Band)** - 时间弹性带算法
- **MPPI** - 模型预测路径积分

4 环境准备

4.1 系统要求

```
# Ubuntu 24.04 + ROS2
OS: Ubuntu 24.04 LTS
ROS: ROS2
Python: 3.10+
```

4.2 安装Navigation2

```
# 安装完整的Navigation2包
sudo apt update
sudo apt install ros-jazzy-navigation2 ros-jazzy-nav2-bringup

# 安装TurtleBot3相关包
sudo apt install ros-jazzy-turtlebot3-gazebo
sudo apt install ros-jazzy-turtlebot3-navigation2

# 安装SLAM工具
sudo apt install ros-jazzy-cartographer-ros
sudo apt install ros-jazzy-slam-toolbox
```



```
# 安装可视化工具
sudo apt install ros-jazzy-rviz2
```

4.3 工作空间设置

```
# 创建工作空间
mkdir -p ~/nav2_ws/src
cd ~/nav2_ws/src

# 克隆必要的包（如果需要自定义）
git clone https://github.com/ros-planning/navigation2.git
git clone https://github.com/ROBOTIS-GIT/turtlebot3.git

# 编译工作空间
cd ~/nav2_ws
colcon build --symlink-install

# 设置环境变量
echo "source ~/nav2_ws/install/setup.bash" >> ~/.bashrc
echo "export TURTLEBOT3_MODEL=waffle" >> ~/.bashrc
source ~/.bashrc
```

4.4 基础配置

1. 机器人配置文件

TurtleBot3配置示例 (nav2_params.yaml)

```
bt_navigator:
  ros__parameters:
    use_sim_time: True
    global_frame: map
    robot_base_frame: base_link
    odom_topic: /odom
    bt_loop_duration: 10
    default_server_timeout: 20

    # 行为树文件路径
    default_nav_to_pose_bt_xml:

/opt/ros/jazzy/share/nav2_bt_navigator/behavior_trees/navigate_to_pose_w_replanning_and_recovery.xml

  plugin_lib_names:
    - nav2_compute_path_to_pose_action_bt_node
    - nav2_compute_path_through_poses_action_bt_node
    - nav2_follow_path_action_bt_node
    - nav2_back_up_action_bt_node
    - nav2_spin_action_bt_node
```

- nav2_wait_action_bt_node
- nav2_clear_costmap_service_bt_node

planner_server:

ros__parameters:

use_sim_time: True

planner_plugins: ["GridBased"]

GridBased:

plugin: "nav2_navfn_planner/NavfnPlanner"

tolerance: 0.5

use_astar: false

allow_unknown: true

controller_server:

ros__parameters:

use_sim_time: True

controller_frequency: 20.0

min_x_velocity_threshold: 0.001

min_y_velocity_threshold: 0.5

min_theta_velocity_threshold: 0.001

controller_plugins: ["FollowPath"]

FollowPath:

plugin: "dwb_core::DWBLocalPlanner"

debug_trajectory_details: True

min_vel_x: 0.0

min_vel_y: 0.0

max_vel_x: 0.26

max_vel_y: 0.0

max_vel_theta: 1.0

min_speed_xy: 0.0

max_speed_xy: 0.26

min_speed_theta: 0.0

acc_lim_x: 2.5

acc_lim_y: 0.0

acc_lim_theta: 3.2

decel_lim_x: -2.5

decel_lim_y: 0.0

decel_lim_theta: -3.2

代价地图配置

local_costmap:

local_costmap:

ros__parameters:

update_frequency: 5.0

publish_frequency: 2.0

global_frame: odom

robot_base_frame: base_link

use_sim_time: True

rolling_window: true

width: 3

height: 3

resolution: 0.05

```

global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 1.0
      publish_frequency: 1.0
      global_frame: map
      robot_base_frame: base_link
      use_sim_time: True
      robot_radius: 0.22
      resolution: 0.05
      track_unknown_space: true

```

2. 启动文件配置

创建完整的启动文件 (nav2_tutorial1.launch.py):

```

#!/usr/bin/env python3
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration, PathJoinSubstitution
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare

def generate_launch_description():
    # 启动参数
    use_sim_time_arg = DeclareLaunchArgument(
        'use_sim_time', default_value='true',
        description='Use simulation time if true')

    # 包路径
    pkg_nav2_bringup = FindPackageShare('nav2_bringup')
    pkg_turtlebot3_gazebo = FindPackageShare('turtlebot3_gazebo')

    # Gazebo仿真
    gazebo_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([
            PathJoinSubstitution([pkg_turtlebot3_gazebo, 'launch',
'turtlebot3_world.launch.py'])
        ]))

    # Navigation2启动
    nav2_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([
            PathJoinSubstitution([pkg_nav2_bringup, 'launch', 'bringup_launch.py'])
        ]),
        launch_arguments={
            'slam': 'True',
            'use_sim_time': LaunchConfiguration('use_sim_time'),
            'params_file': '/path/to/nav2_params.yaml'

```

```
    }.items())

    return LaunchDescription([
        use_sim_time_arg,
        gazebo_launch,
        nav2_launch
    ])
}
```

4 实践教程

步骤1: 启动基础仿真环境

```
# 终端1: 启动Gazebo仿真世界
export TURTLEBOT3_MODEL=waffle && ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

步骤2: 启动建图

SLAM (simultaneous localization and mapping), 同时定位与建图、同时定位与地图构建。

```
# 终端2: 启动Cartographer SLAM
ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
```

步骤3: 手动遥控建图

```
# 终端3: 启动键盘遥控
ros2 run turtlebot3_teleop teleop_keyboard
```

建图操作指南:

- **w/x**: 前进/后退
- **a/d**: 左转/右转
- **s**: 停止
- 慢慢移动机器人, 让激光雷达扫描整个环境

步骤4: 保存地图

```
# 终端4: 保存地图
cd ~/nav2_ws
mkdir -p maps
ros2 run nav2_map_server map_saver_cli -f maps/my_map

ros2 run nav2_map_server map_saver_cli -f my_map2
```

步骤5：查看地图

```
eog maps/my_map.pgm
```

步骤5: 启动turtlebot3_navigation2

先关闭之前启动的turtlebot3_gazebo，然后重新启动turtlebot3_gazebo

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py use_sim_time:=True  
map:=/home/lqf/nav2_ws/maps/my_map.yaml
```

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py use_sim_time:=True  
map:=/home/lqf/maps/my_map3.yaml
```

- 需要指定地图的绝对路径，要不地图不能正常加载

步骤6: 初始定位

```
# 启动RViz2  
# ros2 launch nav2_bringup rviz_launch.py
```

在RViz中：

1. 点击 "2D Pose Estimate" 工具
2. 在地图上点击机器人的大概位置
3. 拖拽箭头设置机器人朝向

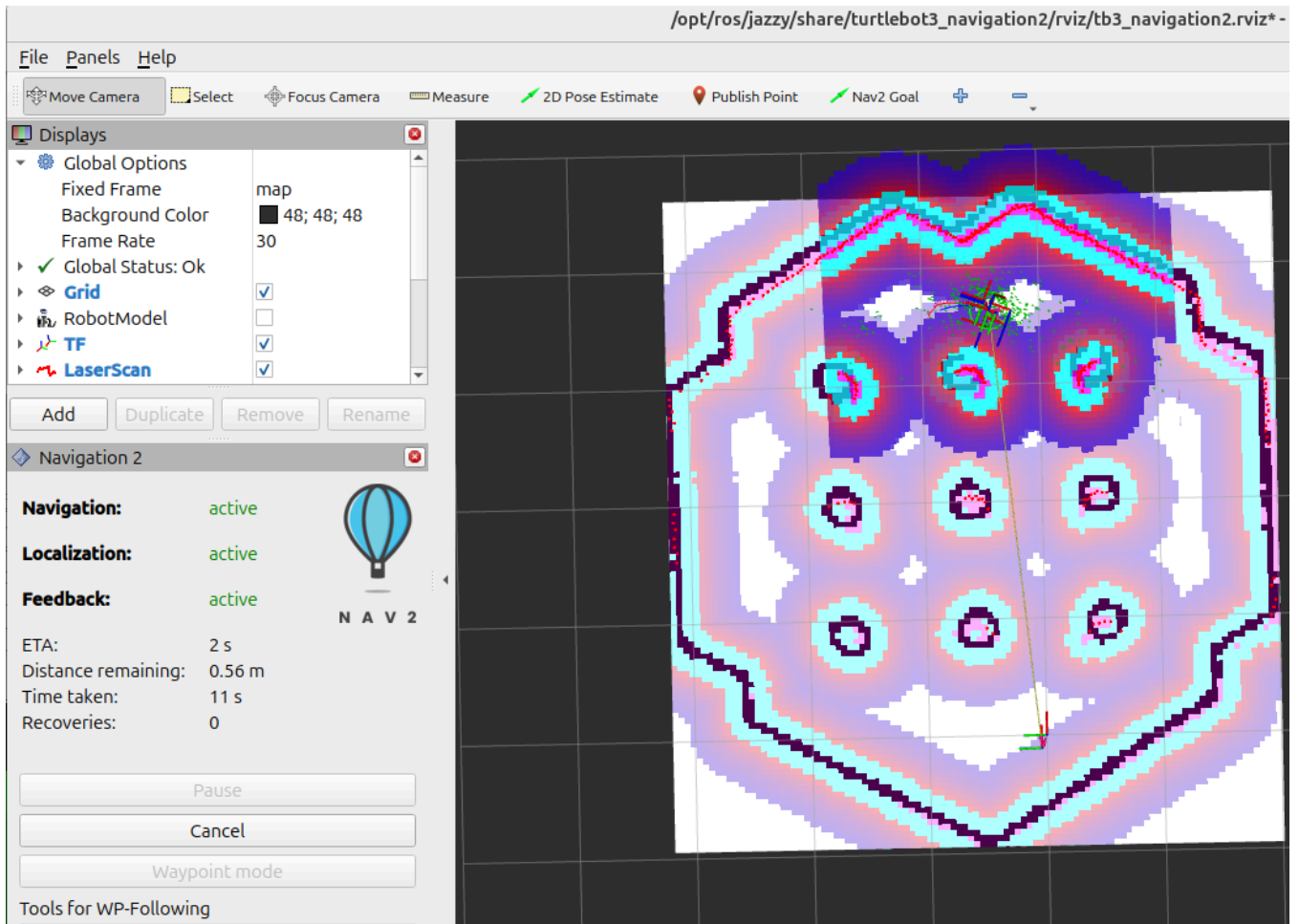
步骤7: 设置导航目标

方法1 - RViz界面：

1. 点击**Nav2 Goal**选择目标点进行导航
2. 在地图上点击目标位置并拖拽设置朝向

方法2 - 命令行：

```
ros2 topic pub --once /goal_pose geometry_msgs/msg/PoseStamped "  
header:  
  frame_id: 'map'  
pose:  
  position: {x: 2.0, y: 1.0, z: 0.0}  
  orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}  
"
```



5 C++ Navigation2导航实现

5.1 创建工作空间

```
# 创建C++工作空间
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src

# 创建
ros2 pkg create nav2_cpp_tutorial --build-type ament_cmake --dependencies rclcpp
rclcpp_action rclcpp_components nav2_msgs geometry_msgs sensor_msgs nav_msgs std_msgs tf2
tf2_ros tf2_geometry_msgs nav2_costmap_2d nav2_util launch_ros
```

###

5.2 项目结构

```
nav2_cpp_tutorial/
├── CMakeLists.txt
├── package.xml
├── include/
│   └── nav2_cpp_tutorial/
│       └── nav2_client.hpp
├── src/
│   ├── nav2_client.cpp
│   └── nav2_simple_test.cpp
├── config/
│   └── nav2_params.yaml
└── launch/
    └── nav2_cpp_demo.launch.py
```

5.3 package.xml配置

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>nav2_cpp_tutorial</name>
  <version>1.0.0</version>
  <description>完整的C++ Navigation2学习教程和示例</description>
  <maintainer email="your.email@example.com">Your Name</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <!-- 核心ROS2依赖 -->
  <depend>rclcpp</depend>
```

```

<depend>roscpp_action</depend>
<depend>roscpp_components</depend>

<!-- Navigation2依赖 -->
<depend>nav2_msgs</depend>
<depend>nav2_util</depend>
<depend>nav2_core</depend>
<depend>nav2_common</depend>
<depend>nav2_costmap_2d</depend>

<!-- 消息和服务依赖 -->
<depend>geometry_msgs</depend>
<depend>sensor_msgs</depend>
<depend>nav_msgs</depend>
<depend>std_msgs</depend>
<depend>std_srvs</depend>
<depend>visualization_msgs</depend>

<!-- TF2依赖 -->
<depend>tf2</depend>
<depend>tf2_ros</depend>
<depend>tf2_geometry_msgs</depend>

<!-- 启动文件依赖 -->
<exec_depend>launch</exec_depend>
<exec_depend>launch_ros</exec_depend>

<!-- 测试依赖 -->
<test_depend>ament_lint_auto</test_depend>
<test_depend>ament_lint_common</test_depend>
<test_depend>ament_cmake_gtest</test_depend>

<export>
  <build_type>ament_cmake</build_type>
</export>
</package>

```

5.4 CMakeLists.txt配置

```

cmake_minimum_required(VERSION 3.8)
project(nav2_cpp_tutorial)

# 默认为C++17
if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# 设置C++17标准
if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 17)
endif()

```


查找依赖包

```
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_action REQUIRED)
find_package(rclcpp_components REQUIRED)
find_package(nav2_msgs REQUIRED)
find_package(nav2_util REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(sensor_msgs REQUIRED)
find_package(nav_msgs REQUIRED)
find_package(std_msgs REQUIRED)
find_package(tf2 REQUIRED)
find_package(tf2_ros REQUIRED)
find_package(tf2_geometry_msgs REQUIRED)
```

包含目录

```
include_directories(include)
```

定义依赖项

```
set(dependencies
  rclcpp
  rclcpp_action
  rclcpp_components
  nav2_msgs
  nav2_util
  geometry_msgs
  sensor_msgs
  nav_msgs
  std_msgs
  tf2
  tf2_ros
  tf2_geometry_msgs
)
```

1. 基础Nav2客户端库

```
add_library(${PROJECT_NAME}_nav2_client SHARED
  src/nav2_client.cpp
)
ament_target_dependencies(${PROJECT_NAME}_nav2_client ${dependencies})
```

2. 简单测试可执行文件

```
add_executable(nav2_simple_test
  src/nav2_simple_test.cpp
)
ament_target_dependencies(nav2_simple_test ${dependencies})
target_link_libraries(nav2_simple_test ${PROJECT_NAME}_nav2_client)
```

安装

```
install(TARGETS
  ${PROJECT_NAME}_nav2_client
  nav2_simple_test
  DESTINATION lib/${PROJECT_NAME}
)
```

```

install(DIRECTORY include/
  DESTINATION include/
)

install(DIRECTORY config/ launch/
  DESTINATION share/${PROJECT_NAME}/
)

ament_package()

```

5.5 核心C++代码实现

Nav2Client类 - 基础客户端头文件

include/nav2_cpp_tutorial/nav2_client.hpp:

```

#ifndef NAV2_CPP_TUTORIAL__NAV2_CLIENT_HPP_
#define NAV2_CPP_TUTORIAL__NAV2_CLIENT_HPP_

#include <memory>
#include <string>
#include <vector>
#include <optional>
#include <mutex>
#include <condition_variable>

#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"
#include "nav2_msgs/action/navigate_to_pose.hpp"
#include "geometry_msgs/msg/pose_stamped.hpp"
#include "geometry_msgs/msg/twist.hpp"
#include "nav_msgs/msg/odometry.hpp"
#include "sensor_msgs/msg/laser_scan.hpp"
#include "std_msgs/msg/string.hpp"
#include "tf2/LinearMath/Quaternion.h"
#include "tf2_geometry_msgs/tf2_geometry_msgs.hpp"

namespace nav2_cpp_tutorial
{
/**
 * @brief 基础的Navigation2客户端类
 *
 * 提供与Navigation2系统交互的基本功能：
 * - 发送导航目标
 * - 监控导航状态
 * - 取消导航任务
 * - 获取机器人位置信息
 */
class Nav2Client : public rclcpp::Node
{

```

```

public:
    using NavigateToPose = nav2_msgs::action::NavigateToPose;
    using GoalHandleNav = rclcpp_action::ClientGoalHandle<NavigateToPose>;

    /**
     * @brief 构造函数
     * @param options 节点选项
     */
    explicit Nav2Client(const rclcpp::NodeOptions & options = rclcpp::NodeOptions());

    /**
     * @brief 析构函数
     */
    virtual ~Nav2Client();

    /**
     * @brief 发送导航目标
     * @param x 目标x坐标 (米)
     * @param y 目标y坐标 (米)
     * @param yaw 目标朝向 (弧度)
     * @param frame_id 坐标系 (默认为"map")
     * @return 是否成功发送目标
     */
    bool sendGoal(double x, double y, double yaw = 0.0, const std::string & frame_id =
"map");

    /**
     * @brief 发送PoseStamped目标
     * @param pose 目标位姿
     * @return 是否成功发送目标
     */
    bool sendGoal(const geometry_msgs::msg::PoseStamped & pose);

    /**
     * @brief 等待导航完成
     * @param timeout_sec 超时时间 (秒)
     * @return 导航是否成功完成
     */
    bool waitForResult(double timeout_sec = 60.0);

    /**
     * @brief 取消当前导航任务
     * @return 是否成功取消
     */
    bool cancelGoal();

    /**
     * @brief 检查Navigation2服务是否可用
     * @param timeout_sec 等待超时时间
     * @return 服务是否可用
     */
    bool isServerReady(double timeout_sec = 5.0);

```

```

/**
 * @brief 获取当前机器人位置
 * @return 当前位置 (如果有效)
 */
std::optional<geometry_msgs::msg::Pose> getCurrentPose() const;

/**
 * @brief 获取当前导航状态
 * @return 状态字符串
 */
std::string getNavigationStatus() const;

/**
 * @brief 创建PoseStamped消息
 * @param x x坐标
 * @param y y坐标
 * @param yaw 朝向
 * @param frame_id 坐标系
 * @return PoseStamped消息
 */
geometry_msgs::msg::PoseStamped createPoseStamped(
    double x, double y, double yaw, const std::string & frame_id = "map");

protected:
/**
 * @brief 目标响应回调
 */
void goalResponseCallback(const GoalHandleNav::SharedPtr & goal_handle);

/**
 * @brief 导航反馈回调
 */
void feedbackCallback(
    GoalHandleNav::SharedPtr,
    const std::shared_ptr<const NavigateToPose::Feedback> feedback);

/**
 * @brief 导航结果回调
 */
void resultCallback(const GoalHandleNav::WrappedResult & result);

/**
 * @brief 里程计回调
 */
void odomCallback(const nav_msgs::msg::Odometry::SharedPtr msg);

/**
 * @brief 激光雷达回调
 */
void laserCallback(const sensor_msgs::msg::LaserScan::SharedPtr msg);

private:
    // Navigation2动作客户端

```

```

rclcpp_action::Client<NavigateToPose>::SharedPtr nav_to_pose_client_;

// 发布者
rclcpp::Publisher<std_msgs::msg::String>::SharedPtr status_publisher_;

// 订阅者
rclcpp::Subscription<nav_msgs::msg::Odometry>::SharedPtr odom_subscriber_;
rclcpp::Subscription<sensor_msgs::msg::LaserScan>::SharedPtr laser_subscriber_;

// 当前状态
GoalHandleNav::SharedPtr current_goal_handle_;
geometry_msgs::msg::Pose current_pose_;
sensor_msgs::msg::LaserScan current_laser_;
std::string navigation_status_;

// 同步原语
mutable std::mutex status_mutex_;
std::condition_variable result_received_;
bool goal_done_;
bool goal_success_;

// 日志前缀
static constexpr char LOG_PREFIX[] = "Nav2Client";
};

} // namespace nav2_cpp_tutorial

#endif // NAV2_CPP_TUTORIAL__NAV2_CLIENT_HPP_

```

Nav2Client类实现

src/nav2_client.cpp 的关键实现部分：

主要功能：

- **连接管理**: 自动连接到Navigation2动作服务器
- **目标发送**: 支持坐标和PoseStamped两种方式
- **状态监控**: 实时反馈导航进度和状态
- **结果等待**: 同步和异步等待导航完成
- **取消功能**: 可随时取消正在进行的导航

C++测试程序

src/nav2_simple_test.cpp 完整的测试程序：

```

#include "nav2_cpp_tutorial/nav2_client.hpp"
#include <iostream>
#include <thread>

using namespace nav2_cpp_tutorial;

```

```

/**
 * @brief C++ Navigation2简单测试程序
 *
 * 演示如何使用Nav2Client类进行基础导航操作：
 * 1. 连接到Navigation2系统
 * 2. 发送导航目标
 * 3. 监控导航状态
 * 4. 等待导航完成
 */

void printTestHeader(const std::string& test_name, int test_num, int total_tests)
{
    std::cout << "\n🔧 测试 " << test_num << "/" << total_tests << ": " << test_name <<
std::endl;
    std::cout << "—————" << std::endl;
}

void printTestResult(bool success, const std::string& message = "")
{
    if (success) {
        std::cout << "✅ 测试成功";
    } else {
        std::cout << "❌ 测试失败";
    }

    if (!message.empty()) {
        std::cout << " - " << message;
    }
    std::cout << std::endl;
}

int main(int argc, char** argv)
{
    rclcpp::init(argc, argv);

    std::cout << "\n🔧 ===== Navigation2 C++基础功能测试 =====" << std::endl;
    std::cout << "请确保已启动完整的Navigation2系统!" << std::endl;
    std::cout << "\n启动命令示例:" << std::endl;
    std::cout << "  ros2 launch turtlebot3_navigation2 navigation2.launch.py" << std::endl;
    std::cout << "  或者使用您项目中的启动文件" << std::endl;

    std::cout << "\n按Enter键继续...";
    std::cin.get();

    try {
        // 创建Nav2客户端
        auto nav2_client = std::make_shared<Nav2Client>();

        // 在后台运行节点
        std::thread spin_thread([nav2_client]() {
            rclcpp::spin(nav2_client);
        });
    }
}

```

```

// 等待系统就绪
std::cout << "\n⌚ 等待Navigation2系统就绪..." << std::endl;
if (!nav2_client->isServerReady(10.0)) {
    std::cout << "❌ Navigation2系统未就绪, 请检查启动配置" << std::endl;
    rclcpp::shutdown();
    spin_thread.join();
    return 1;
}
std::cout << "✅ Navigation2系统已就绪" << std::endl;

// 测试点列表 (x, y, yaw, 描述)
struct TestPoint {
    double x, y, yaw;
    std::string description;
};

std::vector<TestPoint> test_points = {
    {1.0, 0.0, 0.0, "前方1米处"},
    {1.0, 1.0, 1.57, "右前方, 朝北"},
    {0.0, 1.0, 3.14, "右侧, 朝西"},
    {0.0, 0.0, 0.0, "返回原点"}
};

int success_count = 0;
int total_tests = test_points.size();

for (size_t i = 0; i < test_points.size(); ++i) {
    const auto& point = test_points[i];
    printTestHeader(point.description, i + 1, total_tests);

    std::cout << "📍 目标坐标: (" << point.x << ", " << point.y
        << ", " << (point.yaw * 180.0 / M_PI) << "°)" << std::endl;

    // 发送目标
    bool goal_sent = nav2_client->sendGoal(point.x, point.y, point.yaw);
    if (!goal_sent) {
        printTestResult(false, "无法发送目标");
        continue;
    }

    std::cout << "⌚ 等待导航完成..." << std::endl;

    // 等待导航完成 (60秒超时)
    bool navigation_success = nav2_client->waitForResult(60.0);

    if (navigation_success) {
        success_count++;
        printTestResult(true, "导航成功完成");
    } else {
        printTestResult(false, "导航失败或超时");
    }

    // 短暂休息

```

```

        std::this_thread::sleep_for(std::chrono::seconds(2));
    }

    // 测试结果总结
    std::cout << "\n📊 ===== 测试结果总结 =====" << std::endl;
    std::cout << "总测试数: " << total_tests << std::endl;
    std::cout << "成功测试: " << success_count << std::endl;
    std::cout << "成功率: " << (100.0 * success_count / total_tests) << "%" << std::endl;

    if (success_count == total_tests) {
        std::cout << "\n🎉 所有测试通过! Navigation2系统工作正常。" << std::endl;
    } else {
        std::cout << "\n⚠️ 部分测试失败, 请检查系统配置。" << std::endl;
    }

    // 清理
    rclcpp::shutdown();
    spin_thread.join();

} catch (const std::exception& e) {
    std::cerr << "❌ 测试过程中发生异常: " << e.what() << std::endl;
    rclcpp::shutdown();
    return 1;
}

std::cout << "\n🏁 测试程序结束" << std::endl;
return 0;
}

```

配置文件

config/nav2_params.yaml 简化的Navigation2参数配置:

```

# Navigation2参数配置文件 - 针对C++教程优化

bt_navigator:
  ros__parameters:
    use_sim_time: True
    global_frame: map
    robot_base_frame: base_link
    odom_topic: /odom
    bt_loop_duration: 10
    default_server_timeout: 20
    plugin_lib_names:
      - nav2_compute_path_to_pose_action_bt_node
      - nav2_follow_path_action_bt_node
      - nav2_back_up_action_bt_node
      - nav2_spin_action_bt_node
      - nav2_wait_action_bt_node
      - nav2_clear_costmap_service_bt_node

controller_server:

```



```

ros__parameters:
  use_sim_time: True
  controller_frequency: 20.0
  controller_plugins: ["FollowPath"]

  FollowPath:
    plugin: "dwb_core::DWBLocalPlanner"
    max_vel_x: 0.26
    max_vel_theta: 1.0
    acc_lim_x: 2.5
    acc_lim_theta: 3.2

planner_server:
  ros__parameters:
    use_sim_time: True
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner"
      tolerance: 0.5
      use_astar: false
      allow_unknown: true

local_costmap:
  local_costmap:
    ros__parameters:
      use_sim_time: True
      global_frame: odom
      robot_base_frame: base_link
      rolling_window: true
      width: 3
      height: 3
      resolution: 0.05
      robot_radius: 0.22

global_costmap:
  global_costmap:
    ros__parameters:
      use_sim_time: True
      global_frame: map
      robot_base_frame: base_link
      robot_radius: 0.22
      resolution: 0.05

```

启动文件

`launch/nav2_cpp_demo.launch.py` 完整的演示启动文件：

```

#!/usr/bin/env python3

from launch import LaunchDescription

```

```

from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, ExecuteProcess, TimerAction
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration, PathJoinSubstitution
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare

def generate_launch_description():
    # 声明启动参数
    use_sim_time_arg = DeclareLaunchArgument(
        'use_sim_time', default_value='true', description='使用仿真时间')

    slam_arg = DeclareLaunchArgument(
        'slam', default_value='true', description='是否启动SLAM建图')

    rviz_arg = DeclareLaunchArgument(
        'rviz', default_value='true', description='是否启动RViz2可视化')

    run_test_arg = DeclareLaunchArgument(
        'run_test', default_value='false', description='是否自动运行测试程序')

    # 获取包路径
    pkg_turtlebot3_gazebo = FindPackageShare('turtlebot3_gazebo')
    pkg_nav2_bringup = FindPackageShare('nav2_bringup')
    pkg_this = FindPackageShare('nav2_cpp_tutorial')

    # 1. 启动TurtleBot3 Gazebo仿真
    gazebo_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([
            PathJoinSubstitution([pkg_turtlebot3_gazebo, 'launch',
            'turtlebot3_world.launch.py'])
        ]))

    # 2. 启动Nav2 Bringup (完整版本, 包含SLAM)
    nav2_bringup_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([
            PathJoinSubstitution([pkg_nav2_bringup, 'launch', 'bringup_launch.py'])
        ]),
        launch_arguments={
            'slam': LaunchConfiguration('slam'),
            'use_sim_time': LaunchConfiguration('use_sim_time'),
            'params_file': PathJoinSubstitution([pkg_this, 'config', 'nav2_params.yaml']),
            'use_rviz': LaunchConfiguration('rviz')
        }.items()
    )

    # 3. C++测试节点 (延迟启动)
    test_node = TimerAction(
        period=10.0, # 10秒后启动测试
        actions=[
            Node(
                package='nav2_cpp_tutorial',

```

```

        executable='nav2_simple_test',
        name='nav2_cpp_test',
        output='screen',
        condition=IfCondition(LaunchConfiguration('run_test'))
    )
]
)

return LaunchDescription([
    # 参数声明
    use_sim_time_arg,
    slam_arg,
    rviz_arg,
    run_test_arg,

    # 核心启动
    gazebo_launch,
    nav2_bringup_launch,
    test_node,

    # 信息打印
    ExecuteProcess(
        cmd=['echo', '@ Navigation2 C++演示系统启动完成!'],
        output='screen'
    ),
])

```

5.6 编译和运行

步骤1: 安装依赖并编译

```

cd ~/nav2_cpp_ws

# 安装依赖
rosdep install -y --from-paths src --ignore-src --rosdistro jazzy

# 编译
colcon build --symlink-install

cd ~/ros2_ws && colcon build --packages-select nav2_cpp_tutorial --install-base install

# 设置环境
source install/setup.bash

```

步骤2: 启动完整系统

```
# 设置TurtleBot3模型
export TURTLEBOT3_MODEL=waffle

# 启动完整演示系统
ros2 launch nav2_cpp_tutorial nav2_cpp_demo.launch.py
```

步骤3: 运行测试

方法1: 自动测试

```
# 在启动时自动运行测试
ros2 launch nav2_cpp_tutorial nav2_cpp_demo.launch.py run_test:=true
```

方法2: 手动测试

```
# 等待系统完全启动后，在新终端运行
ros2 run nav2_cpp_tutorial nav2_simple_test
```

方法3: 交互式测试

```
# 在RViz中使用"2D Goal Pose"工具设置目标
# 或使用命令行发送目标
ros2 topic pub --once /goal_pose geometry_msgs/msg/PoseStamped "
header:
  frame_id: 'map'
pose:
  position: {x: 2.0, y: 1.0, z: 0.0}
  orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}"
```
